book-qr-manager 学習ノート

学習日：2026年8月29日
テーマ：NestJSのModule・Service・RepositoryとTypeORMの基本

1. 今日やったこと

今日は、`Book` と `Location` のEntityをPostgreSQLのテーブルとして作成できた状態から、NestJSでデータベースを操作するための構成を学習しました。

- Repositoryとは何か
- `TypeOrmModule.forFeature()` の役割
- Serviceと`@Injectable()`の役割
- `@InjectRepository()`によるRepositoryの注入
- Moduleと`providers`の役割
- `BooksModule` / `LocationsModule` の作成
- `AppModule`へのModule登録
- NestJS → TypeORM → PostgreSQL のつながり

2. Repositoryとは？

Repositoryは、**Entityを使ってデータベースを操作するための窓口**です。

EntityはDBテーブルの構造をTypeScriptのクラスとして表しますが、Entity自身が検索や保存をするわけではありません。

```
Book Entity
    ↓
Book Repository
    ↓
PostgreSQL
```

例えば今後、次のように使います。

```
this.bookRepository.find();       // 検索
this.bookRepository.save(book);   // 保存
```

3. `TypeOrmModule.forFeature()`とは？

ServiceからRepositoryを使うため、NestJSに「このModuleではこのEntityのRepositoryを使う」と登録します。

```
TypeOrmModule.forFeature([Book])
```

BooksModuleでは、

```
@Module({
  imports: [TypeOrmModule.forFeature([Book])],
})
export class BooksModule {}
```

Locationなら、

```
TypeOrmModule.forFeature([Location])
```

です。

覚え方： `forFeature([Book])` = 「この機能ではBook Repositoryを使う」。

4. Serviceとは？

Serviceは、アプリケーションの**処理を担当するクラス**です。

書籍管理なら「本を登録する」「検索する」「更新する」「削除する」といった処理を書いていきます。

```
Controller
    ↓ HTTPリクエストを受け取る
Service
    ↓ 実際の処理を行う
Repository
    ↓ DBを操作する
Entity
    ↓ DBテーブルの構造を表す
PostgreSQL
```

5. `@Injectable()`とは？

Serviceには、

```
@Injectable()
export class BooksService {}
```

と書きます。

`@Injectable()`は、このクラスを**NestJSのDI（依存性注入）の仕組みで管理できるようにする**ためのデコレーターです。

6. RepositoryをServiceへ注入する

```
@Injectable()
export class BooksService {
  constructor(
    @InjectRepository(Book)
    private readonly bookRepository: Repository<Book>,
  ) {}
}
```

`@InjectRepository(Book)` は、

> Book Entity用のRepositoryをここへ注入してください

という意味です。

`private readonly bookRepository: Repository<Book>` によって、Service内で`this.bookRepository`として利用できます。

```
TypeOrmModule.forFeature([Book])
        ↓
Book Repositoryを利用可能にする
        ↓
@InjectRepository(Book)
        ↓
BooksServiceへ注入
        ↓
this.bookRepositoryでDB操作
```

7. Moduleとは？

Moduleは、NestJSで**関連する機能をひとまとめにする単位**です。

現在のBook機能は、

```
src/books/
├── book.entity.ts
├── books.module.ts
└── books.service.ts
```

今後は、

```
books/
├── book.entity.ts
├── books.controller.ts
├── books.service.ts
├── books.module.ts
└── dto/
```

```
    ├── create-book.dto.ts
    └── update-book.dto.ts
```

のように成長していきます。

Locationも同じ考え方です。

```
books/      → Bookに関係する機能
locations/  → Locationに関係する機能
```

8. `providers`とは？

```ts
@Module({
  imports: [TypeOrmModule.forFeature([Book])],
  providers: [BooksService],
})
export class BooksModule {}
```

`providers: [BooksService]` は、**BooksServiceをNestJSに管理してもらうための登録**です。

```
BooksModule
├── imports
│     └── Book Repository
└── providers
      └── BooksService
```

9. 現在のBooksModule

```ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Book } from './book.entity';
import { BooksService } from './books.service';

@Module({
  imports: [TypeOrmModule.forFeature([Book])],
  providers: [BooksService],
})
export class BooksModule {}
```

10. 現在のBooksService

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Book } from './book.entity';

@Injectable()
export class BooksService {
  constructor(
    @InjectRepository(Book)
    private readonly bookRepository: Repository<Book>,
  ) {}
}
```

11. Location側で復習したこと

Bookで学習したパターンを利用し、Location側は自分で実装しました。

```
@Module({
  imports: [TypeOrmModule.forFeature([Location])],
  providers: [LocationsService],
})
export class LocationsModule {}
```

```
@Injectable()
export class LocationsService {
  constructor(
    @InjectRepository(Location)
    private readonly locationRepository: Repository<Location>,
  ) {}
}
```

BookとLocationで基本構造は同じです。

12. AppModuleへの登録

作成したModuleは`AppModule`の`imports`にも登録します。

```
@Module({
  imports: [
    // DB接続設定など
    BooksModule,
    LocationsModule,
  ],
})
export class AppModule {}
```

これにより、

```
AppModule
├── BooksModule
└── LocationsModule
```

という構成になります。

13. forRoot() と forFeature() の違い

ここは混乱しやすい重要ポイントです。

forRoot()

```
TypeOrmModule.forRoot({
  type: 'postgres',
  host: process.env.DB_HOST,
  // ...
})
```

TypeORMとPostgreSQLをどう接続するかを設定します。

forFeature()

```
TypeOrmModule.forFeature([Book])
```

そのModuleでどのEntityのRepositoryを使うかを登録します。

```
forRoot()
  → PostgreSQLにどう接続する？

forFeature([Book])
  → BooksModuleではBook Repositoryを使う
```

14. 今日の全体像

```
AppModule
│
├── TypeOrmModule.forRoot(...)
│       ↓
│   PostgreSQL接続
│
├── BooksModule
│   ├── TypeOrmModule.forFeature([Book])
│   │       ↓
│   │   Book Repository
│   └── BooksService
│           ↓
│       @InjectRepository(Book)
│
└── LocationsModule
    ├── TypeOrmModule.forFeature([Location])
    │       ↓
    │   Location Repository
    └── LocationsService
            ↓
        @InjectRepository(Location)
```

15. 今日の重要ポイント

1. **Entity**：DBテーブルの構造を表現する。
2. **Repository**：Entityを使ってDBを検索・保存・更新・削除する。
3. **Service**：アプリケーションの処理を書く。
4. **Module**：関連する機能をまとめ、NestJSへ登録する。
5. `forFeature()`：そのModuleで使用するEntityのRepositoryを登録する。

迷ったときは、

```
Controller
    ↓
Service
    ↓
Repository
    ↓
Entity
    ↓
PostgreSQL
```

という流れを思い出します。

16. 次回やること

次回はLocationを実際にPostgreSQLへ登録する処理へ進む予定です。

1. `CreateLocationDto`を作る
2. DTOとは何かを理解する
3. `repository.create()`を理解する
4. `repository.save()`を理解する
5. `LocationsService`に登録処理を作る

`repository.create()`はEntityオブジェクトを作成する処理で、**この時点ではDBへ保存されません。**

`repository.save()`で初めてDBへの保存処理が行われます。

復習用ひとこと

Moduleで機能をまとめ、Serviceで処理を行い、RepositoryをEntityをDBへ保存・検索する。

今日の段階では、まずこの流れをイメージできればOKです。